Complexity Analysis

Assume you have two recursive algorithms to solve the same problem. Which one would you prefer? The answer is obvious; you will nominate the one with lower asymptotic time and space complexity bounds.

But, how to analyze the recursive algorithms for asymptotic time and space complexity bounds? Are there any classical methods to that? Let's find out!

Note: This section assumes that you have a fair knowledge of asymptotic bounds (e.g., Big-O, omega, and theta notations).

Asymptotic time complexity for recursive algorithms

In contrast to iterative algorithms, asymptotic time complexity analysis for recursive algorithms is not very straightforward. The recursive overhead, non-linear behavior, and the need to solve recurrence relations (we will discuss later in the section) makes the analysis difficult.

Therefore, many experts have proposed some standard techniques to evaluate the recursive algorithms and approximate the asymptotic time bounds. These techniques include the recursion tree method, substitution method, and master theorem.

Before starting a discussion on these complexity analysis techniques, let's discuss an important pre-requisite - Recurrence relation.

Recurrence relation

Recurrence relation/ equation is a way to mathematically capture the approximate behavior of a recursive algorithm. Having a recurrence equation for a recursive algorithm often comes handy to estimate overall asymptotic time complexity of this algorithm.

Here is a generalized form of recurrence equation: $T(n) = aT\left(\frac{n}{b}\right) + F(n)$, where:

- $T(n)$ represents the time complexity of the algorithm for an input size of n.
- a represents the number of subproblems that breaking a larger problem will result in.
- $\left(\frac{n}{b}\right)$ is the size of each subproblem that resulted in breaking a larger problem of size n.
- $F(n)$ is the asymptotic time bound for dividing and then conquering/merging a problem of size n. Please note if n approaches the base case, we say the $F(n)$ is bounded by $O(1)$.

For example, the recurrence $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$ is a mathematical approximation of any recursive algorithm that fulfills the following conditions:

1. It divides the larger problem into two subproblems
2. The size of each subproblem is the exact half the larger one
3. For dividing and then conquering the problem of size n, the algorithm takes $O(n)$ time.

Enough on the recurrence equations, let's now move on to start discussion with our first complexity analysis technique - the recursion tree method.

1. Recursion tree method

The recursion tree method is a technique used to analyze the time complexity of recursive algorithms. It involves representing the recursive calls of an algorithm as a tree structure, known as a recursion tree, where each node represents a recursive call.

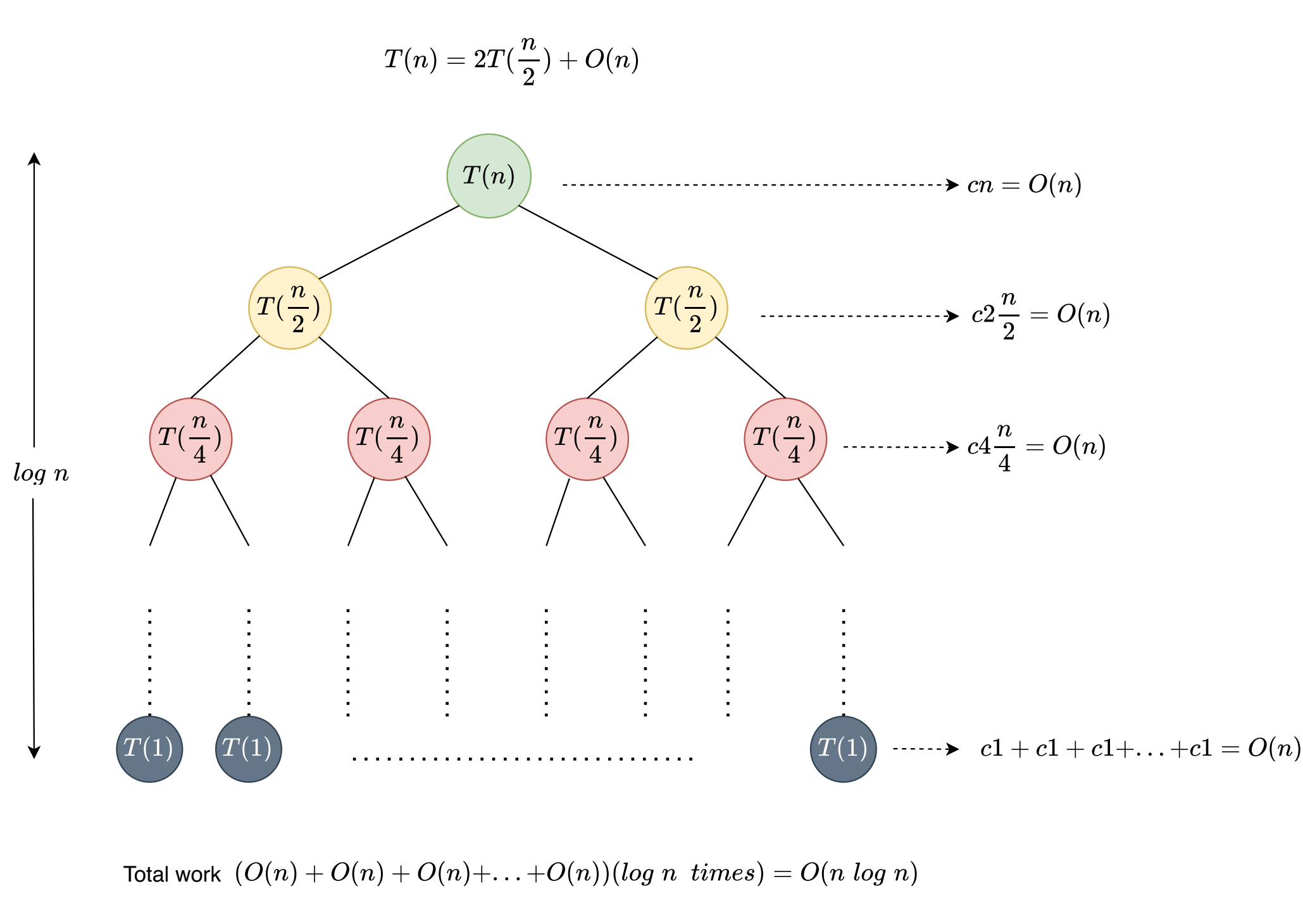
The recursion tree visualizes the number of recursive calls made and the work done at each level of recursion. Here's a step-by-step explanation of how the recursion tree method works:

1. Start by drawing a tree with a root node representing the initial call to the recursive procedure.
2. For each recursive call, create child nodes that represent subsequent recursive calls made within that call. Repeat this process until you reach the base case(s) of the recursion.
3. Assign a work or cost value to each node, representing the amount of work done in that recursive call.
4. Analyze the tree by calculating the total work done at each level of recursion. This can be done by summing up the work values of all the nodes at each level.
5. Determine the depth of the recursion tree, which represents the number of levels in the tree. This is often related to the input size or the number of recursive calls made.
6. Finally, analyze the total work done as a function of the recursion depth. This helps in determining the time complexity of the algorithm.

By analyzing the recursion tree and observing patterns in the work done at each level, you can gain insights into the time complexity of the recursive algorithm. This method is particularly useful for algorithms with multiple recursive calls or when the time complexity is not immediately apparent.

Example

Assume a divide & conquer algorithm that fulfills the **recurrence equation**: $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$, where $T(n)$ represents the time complexity of the algorithm for an input size of `n`. The following figure presents a corresponding recursion tree and the overall asymptotic time complexity calculation for this algorithm.



Recursion Tree

2. Substitution method

The substitution method is a technique used to analyze the time complexity of algorithms by making educated guesses about the time complexity and then proving them using mathematical induction.

Here's how the Substitution Method works:

1. Start by making an educated guess about the time complexity of the algorithm.
2. Assume that the guess is correct and solve the recurrence relation using mathematical induction.

   Base case: Verify that the guess holds for the recurrence relation's base case(s). Typically, this involves checking the time complexity for small input sizes.

   Inductive hypothesis: Assume that the guess holds for all input sizes smaller than n.

   Inductive step: Using the assumption from the inductive hypothesis, prove that the guess holds for the input size n.

3. Once the recurrence relation is solved, you obtain a closed-form solution, which represents the algorithm's time complexity.

Here's an example to illustrate the Substitution Method.

Consider the following pseudo-code of the Merge Sort algorithm:

```
mergeSort(arr[], low, high)
    if low < high
        mid = (low + high) / 2
        mergeSort(arr, low, mid)
        mergeSort(arr, mid + 1, high)
        merge(arr, low, mid, high)
```

In the above pseudocode, `mergeSort` is the main function that performs the Merge Sort algorithm. It recursively divides the array into smaller halves until the base case is reached (when `low` is no longer less than `high`). The `merge` procedure is responsible for merging the sorted halves back together.

The Recurrence of the above pseudo-code is as follows:

$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

We want to determine the time complexity of using the Substitution Method. Let's make an educated guess that the time complexity is $O(nlogn)$.

To prove this guess, we can use mathematical induction:

1. **Base case**: For small input sizes, such as when the array has only one element, the algorithm performs a constant amount of work. Therefore, the guess holds for the base case.
2. **Inductive hypothesis**: Assume that the guess holds for arrays of size smaller than n.
3. **Inductive step**: We analyze the recurrence relation of Merge Sort. The algorithm divides the array in half and recursively sorts the two halves. The merge operation takes O(n) time.

   Assuming the guess holds for arrays of size n/2, we can express the time complexity of the algorithm as:

   $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

   By the inductive hypothesis, we substitute T(n/2) with the assumed time complexity:

   $T(n) = 2[\frac{n}{2}log(\frac{n}{2})] + O(n)$

   Simplifying further, we have:

   $T(n) = nlog(\frac{n}{2}) + O(n) = nlog(n) - nlog(2) + O(n) = nlogn - n + O(n) = O(nlogn)$

   Thus, we have proved that the time complexity of Merge Sort is $O(nlogn)$.

The Substitution Method allows us to make an educated guess about the time complexity of an algorithm and then mathematically prove it using induction. It provides a systematic approach to analyzing recursive algorithms.

3. Master theorem method

The master theorem is a mathematical tool used to analyze and solve certain types of recurrence relations. It provides a method for determining the asymptotic behavior of functions that satisfy a specific form of the recurrence relation.

The master theorem is typically used for recurrence relations of the following form:

$T(n) = aT\left(\frac{n}{b}\right) + f(n)$

where:

- $T(n)$ represents the time complexity (or recurrence) of an algorithm or function for a problem size of n.
- a ≥ 1 and b > 1 are constants that define the number of recursive subproblems and the size reduction factor, respectively.
- f(n) represents the non-recursive part of the time complexity, which includes any additional work done outside of the recursive calls.

The master theorem provides a formula for solving such recurrence relations and determining the asymptotic time complexity of the algorithm or function. It has three cases based on the relationship between the recursive and non-recursive parts:

Case 1: If $f(n) = O(n^c)$ for some constant $c < log_b(a)$, then the time complexity can be expressed as: $T(n) = \Theta(n^{log_b(a)})$.

Case 2: If $f(n) = \Theta(n^c log(n))$ for some constants $c \geq 0$, and if a recursive subproblem dominates the non-recursive part, i.e., $aT(\frac{n}{b}) \geq f(n)$ for sufficiently large n, then the time complexity can be expressed as $T(n) = \Theta(n^{log_b(a)} log(n))$.

Case 3: If $f(n) = \Omega(n^c)$ for some constant $c > log_b(a)$, and if the non-recursive part dominates the recursive subproblems, i.e., $aT(\frac{n}{b}) \leq f(n)$ for sufficiently large n, then the time complexity can be expressed as: $T(n) = \Theta(f(n))$.

To apply the master theorem, you must identify which case applies to your recurrence relation by comparing the growth rates of $f(n)$ and $n^{log_b(a)}$. Once you determine the case, you can directly obtain the asymptotic time complexity using the corresponding formula.

It's important to note that the master theorem can only be applied to recurrence relations that adhere to the specific form mentioned earlier. Recurrence relations with different structures or dependencies may require alternative methods for analysis and solution.

The master theorem is particularly useful for analyzing the time complexity of algorithms that divide a problem into smaller subproblems and recursively solve them. It allows for quickly determining the asymptotic behavior without explicitly solving the recurrence relation.

Let's work through a few examples to illustrate the application of the master theorem.

Example 1: Consider the following recurrence relation: $T(n) = 9T(\frac{n}{3}) + n^2$.

In this case, we have a = 9, b = 3, and $f(n) = n^2$.

To apply the master theorem, we compare the growth rate of $f(n) = n^2$ with $n^{log_b(a)} = n^{log_3(9)} = n^2$.

Since $f(n) = n^2$ matches the growth rate of $n^{log_b(a)} = n^2$, we are in Case 1 of the master theorem.

Therefore, the time complexity can be expressed as: $T(n) = \Theta(n^{log_b(a)}) = \Theta(n^2)$.

Example 2: Consider the following recurrence relation: $T(n) = 2T(\frac{n}{2}) + n^3$.

In this case, we have a = 2, b = 2, and $f(n) = n^3$.

Comparing the growth rate of $f(n) = n^3$ with $n^{log_b(a)} = n^{log_2(2)} = n^1 = n$, we find that f(n) grows faster than $n^{log_b(a)}$.

Therefore, we are in **Case 3** of the master theorem.

Thus, the time complexity can be expressed as: $T(n) = \Theta(f(n)) = \Theta(n^3)$.

Example 3: Consider the following recurrence relation: $T(n) = 4T(\frac{n}{2}) + n^2$.

In this case, we have a = 4, b = 2, and $f(n) = n^2$.

Comparing the growth rate of $f(n) = n^2 logn$ with $n^{log_b(a)} = n^{log_2(4)} = n^2$, we find that f(n) is of the same growth rate as $n^{log_b(a)}$.

Therefore, we are in **Case 2** of the master theorem.

Hence, the time complexity can be expressed as: $T(n) = \Theta(n^{log_b(a)} logn) = \Theta(n^2 logn)$.

In each of these examples, the master theorem helps us determine the asymptotic time complexity of the given recurrence relation without explicitly solving it. By comparing the growth rates of the recursive and non-recursive parts, we can quickly identify the appropriate case and obtain the time complexity.

Comparison table

The below table summarizes the key aspects of the Recursion Tree method, Substitution method, and Master Theorem as techniques for analyzing the complexity of recursive algorithms:

Technique	Recursion Tree Method	Substitution Method	Master Theorem
Overview	Constructs a tree to visualize recursive calls	Guesses an upper bound and proves it by mathematical induction	Provides formulas for the complexity based on recurrence relation form
Applicability	Useful for analyzing algorithms with varying subproblem sizes and branching factor	Applicable when there is an intuitive guess for the complexity	Useful for divide-and-conquer algorithms with equal subproblem sizes
Complexity Type	Both time and space complexity can be analyzed	Mostly used for time complexity analysis	Primarily used for time complexity analysis
Recursive Structure	Explores different branches of the recursion	Replaces a recursive call with a solution guess	Applies a formula to compute complexity directly
Requirements	Requires analyzing the number of nodes at each level and the work done at each level	Requires a good initial guess and proof by mathematical induction	Requires the recurrence relation to match the specific form
Complexity Analysis	Can be tedious for complex recursion trees	Easier to perform for simple recurrence relations	Provides a concise formula for complexity analysis
Limitations	Not always applicable for complex recursion structures	Limited to cases where an accurate guess can be made	Applicable only to specific recurrence relation forms
Examples	Tree traversals, merge sort, quicksort	Fibonacci sequence, binary search	Merge sort, binary search, matrix multiplication

Space complexity analysis for recursive algorithms

Space complexity analysis for recursive algorithms involves analyzing the amount of additional space required by the algorithm as a function of the input size. This analysis helps determine how much memory the algorithm needs to execute and how it scales with increasing input sizes.

When analyzing space complexity in recursive algorithms, there are a few key factors to consider:

1. **Recursive stack space**: Recursive algorithms typically rely on function calls that create stack frames to store local variables and the return address. Each function call adds a new stack frame to the call stack, which consumes space. The depth of the recursive calls determines the maximum number of stack frames needed at any given time.
2. **Additional data structures**: Recursive algorithms may create additional data structures or arrays during their execution. The space required by these data structures can contribute to the overall space complexity.

To perform space complexity analysis for recursive algorithms, you can follow these steps:

1. **Identify the additional space used by each recursive call**: Determine the amount of space used by the recursive calls themselves, including the space required for each stack frame. This includes any local variables, parameters, and return addresses stored in the stack frames.
2. **Determine the number of recursive calls made**: Analyze the algorithm to determine the number of recursive calls made based on the input size. This information helps determine the maximum depth of the recursive calls and, consequently, the maximum number of stack frames needed.
3. **Calculate the space complexity**: Multiply the additional space used by each recursive call by the maximum number of recursive calls to obtain the total space complexity of the algorithm. Consider any additional data structures created during the recursion as well.

It's important to note that the space complexity analysis for recursive algorithms does not include the space used by the input itself. It focuses on the additional space requirements due to recursive calls and any extra data structures created.

Additionally, some recursive algorithms may be optimized to reduce space usage. Techniques like tail recursion, memoization, or iterative versions of the algorithm can often be employed to reduce or eliminate the need for additional stack frames or data structures, resulting in improved space efficiency.

By performing space complexity analysis, you can gain insights into the memory requirements of recursive algorithms and evaluate their efficiency in terms of space usage.

Example

Let's work through an example of analyzing the space complexity of a recursive algorithm. Consider the following example of calculating the factorial of a number using a recursive algorithm:

```
function factorial(n):
    if n ⩽ 1:
        return 1
    else:
        return n * factorial(n-1)
```

To analyze the space complexity of this recursive algorithm, we'll consider the space used by the recursive calls and the stack frames.

1. **Additional space used by each recursive call**: In this algorithm, each recursive call creates a stack frame that stores the local variables `n` and the return address. The space used by each recursive call is constant and does not depend on the input size.
2. **Number of recursive calls**: The number of recursive calls made in this algorithm is `n-1`. Starting from `n` and recursively calling `factorial` with `n-1` until the base case is reached.
3. **Calculate the space complexity**: Since each recursive call uses a constant amount of space, and the maximum depth of the recursion is `n-1`, the space complexity of this algorithm is `O(n)`.

So, the space complexity of the factorial recursive algorithm is linear with respect to the input size `n`.

It's important to note that the space complexity analysis only considers the additional space used by the recursive calls and stack frames. The space used by the input parameter `n` is not included in the space complexity analysis.

In practice, if the recursive algorithm has a large depth of recursion or the input size is large, it may be necessary to consider space optimization techniques such as tail recursion or iterative approaches to reduce the space requirements.

Let's apply what we've learned to real scenarios. In the upcoming sections, we'll solve some actual coding interview questions using recursion.